Convolutional Neural Networks (CNNs)

DSC 340 Applied Machine Learning

Unit: Deep Learning #3

April 25, 2025

The Plan

- Wrap up discussion of ANN training
- Brief overview of CNNs
- Limitations of 1980s style MLPs
- Basic concepts (e.g., max-pooling)
- Team Time (hopefully!)

Backpropagation

Gradient Descent

- Go "downhill"
- Iterative improvement
- Stop when costs stop improving
- Assumption: convex
  - Stochastic gradient descent (SGD)
  - Batch gradient descent (BGD)

Backpropagation (big picture)

- **Initialize** to small random weights
- **Feed forward** X through network
- **Compute cost** J (how far off were we?)
- **Backpropagate** errors to each weight
- **Adjust** weights accordingly
- Each example has cost and gradient
- Repeat for many **epochs** until converges

To clarify

- **Solve gradients analytically** (calculus problem)
  - Or look up in a book, journal paper, or the web!
  - Activation functions carefully chosen to make this process efficient
- Use **data** and **errors** to **compute** **gradients** (e.g., NumPy, BLAS, GPU)
- Find "downhill" in M-dim optimization space
- Add/subtract scalar times gradient to all weights
  - Scalar constant known as the learning rate
- Relies on linear algebra libraries (fast!), not objects and loops (slow!)
  - Vectorized operations

Convolutional Neural Nets (CNNs)





What MLPs "see"

- Idea: Use **fully connected** ANNs for image classification
- How does it work?
- We imagine the first layer is direct inputs
- Intermediate layers capture edges and shapes
- Gradually capture increasing layers of abstraction, finally classify!
- **Turns out this is NOT how fully-connected MLPs work**
- Sees all the neurons all the time
- In a sense, the MLP just memorizes the answers!

Convolutional Neural Networks

- Introduced in 2012
- ImageNet competition: reduced error from 26% to 15%
- Since widely adopted (Facebook, Google, Amazon, Pinterest, …)
- **Sparse network, not FC** [= fully connected]

Problem



What We See

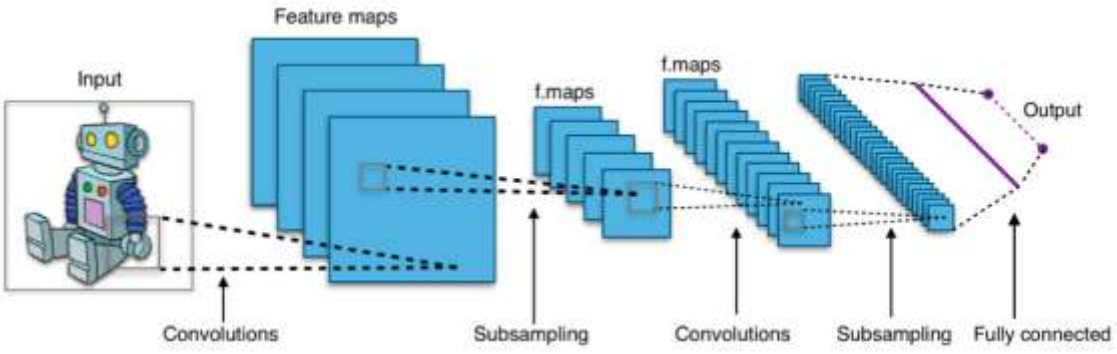
```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 26 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 26 97 17 78 78 96 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48
```

What Computers See

CS meets biology

- Image represented as a tensor (array of matrices)
- Example: 480 x 480 x 3 (one matrix each for RGB channels)
- Values are from 0 to 255 based on intensity of white or RGB channel
- Biological inspiration: cerebral cortex
- Hubel and Wiesel (1962) neurons fired based on orientation of edges
- Successive research: layers with presumably increasing abstraction
- In other words, the way we think neural nets should work!

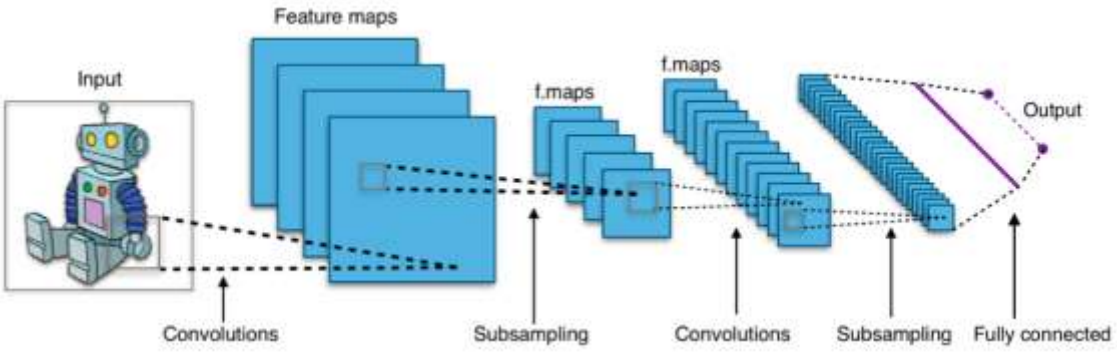
Convolutional Neural Nets (CNNs)



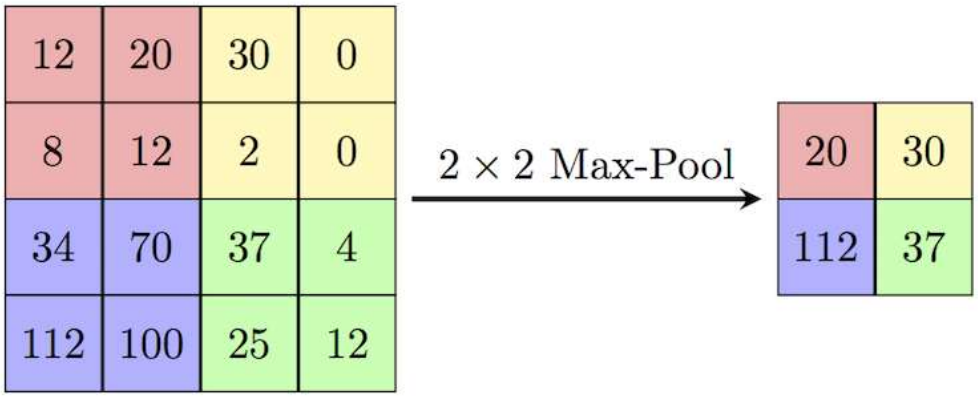
Structure

- Input layer
- Convolutional layers (note: many hidden layers)
  - Filters ("sliding window")
  - Zoom in and out
  - Look at different parts of the image
  - Different levels of brightness
  - Edges and shapes
- Finally a fully connected (FC) layer (again, several hidden layers)
- Output layer classifies image

Convolutional Neural Nets (CNNs)

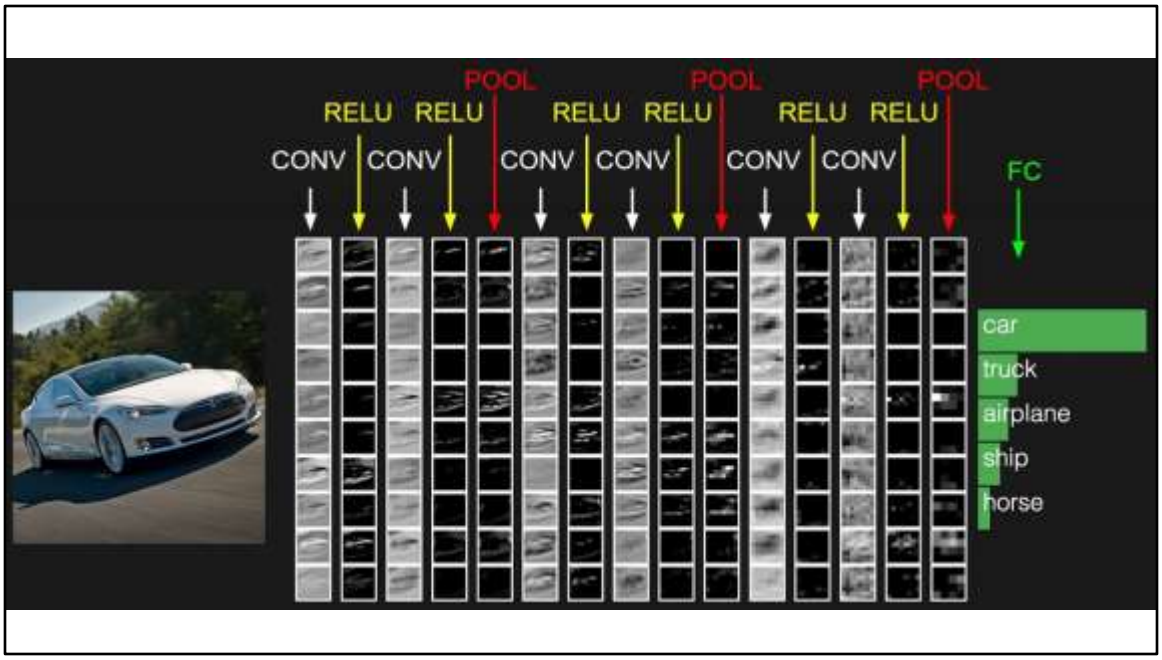


Pooling



Convolution-speak

- "Convolve" = slide window over image
- "Kernel" (or "convolution kernel") = little sliding window
- "Stride" = number of pixels that window shifts each time
- "Padding" = overlaps with other sampled windows
- "Pooling" = combine multiple pixels (e.g., 3x3) into 1
- "Max-pool" (1 pixel gets max of 3x3), average-pool (1 gets mean), …


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

Recap

- Inspired by layers in visual cortex
- Convolutional layers
- Input is a tensor:
  - Shape: (number of images) x (image width) x (image height) x (image depth).
  - Depth: e.g., color channels R,G,B; e.g., k x 24 x 24 x 3
- Convolutional kernels
  - Width and height of kernels are hyper-parameters
  - Apply convolution op to input, passing the result to the next layer
  - Emulates the response of an individual neuron to visual stimuli
- Mask R-CNN (FAIR): https://medium.freecodecamp.org/mask-r-cnn-explained-7f82bec890e3